

A Unified Graph Clustering Network

Renda Han

Tianjin Key Laboratory of Cognitive
Computing and Application, School
of Artificial Intelligence, Tianjin
University
Tianjin, China
rd.h@computer.org

Wenxin Zhang

University of Chinese Academy of
Sciences
Beijing, China
zwxzhang12@163.com

Xiaobao Wang*

Tianjin Key Laboratory of Cognitive
Computing and Application, School
of Artificial Intelligence, Tianjin
University
Tianjin, China
wangxiaobao@tju.edu.cn

Ronghao Fu

Jilin University
Changchun, China
fuh@jlu.edu.cn

Longbiao Wang*

Tianjin Key Laboratory of Cognitive
Computing and Application, School
of Artificial Intelligence, Tianjin
University
Tianjin, China
longbiao_wang@tju.edu.cn

Kaiming Wang

Southeast University
Nanjing, China
220224867@seu.edu.cn

Zeyu Zhang

The Australian National University
Canberra, Australia
steve.zeyu.zhang@outlook.com

Kuntharrgyal Khysru

Qinghai Minzu University
Xining, China
gtj186@tju.edu.cn

Abstract

Clustering is a fundamental task in graph data mining, including both node-level and graph-level clustering. While the former has been extensively explored to capture local structures and features, the latter has gained attention for its ability to capture global relationships and high-level abstractions. However, existing methods often address these two tasks in isolation, which not only wastes computational resources but also fails to fully leverage the knowledge from both levels to improve each other, hindering consistent performance improvement. To this end, we propose a novel Unified Graph Clustering Network called UGCN, which employs both local and global graph information to address node- and graph-level clustering collaboratively. In detail, we design a dual-branch projector that performs joint learning at both node and graph levels. The first branch extracts node-level features and projects them into distinct cluster layers, where the derived prototypes are used to refine graph attributes and highlight clustering-friendly substructures. In parallel, the second branch captures subgraph embeddings and aggregates them into discriminative graph-level representations. We align the two branches through joint contrastive objectives to establish a bidirectional interaction: refined prototypes guide subgraph and graph-level clustering, while graph-level pseudo-labels provide feedback to enhance node-level clustering. Extensive experimental results across seven datasets demonstrate that our method significantly outperforms existing state-of-the-art approaches.

* Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792266>

CCS Concepts

• **Computing methodologies** → **Neural networks**; *Cluster analysis*.

Keywords

Graph Neural Network; Unsupervised Clustering; Graph Representation Learning; Deep Graph Learning

ACM Reference Format:

Renda Han, Xiaobao Wang, Longbiao Wang, Wenxin Zhang, Ronghao Fu, Kaiming Wang, Zeyu Zhang, and Kuntharrgyal Khysru. 2026. A Unified Graph Clustering Network. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792266>

1 Introduction

Graph clustering [9, 23, 24, 30, 32], which aims to group samples based on structural and feature similarities [22, 31, 49], has become a critical task in understanding and improving web-based systems. By effectively grouping graph data, we can enhance various web applications, such as social network analysis [51], multimodal sentiment identification [12, 34], and fake news detection [15, 47, 48]. Currently, graph clustering can be broadly categorized into node-level and graph-level clustering. The former focuses on grouping nodes within a graph based on local patterns without labeled data. Methods such as SCGC [20] and ABR [29] leverage self-supervision contrastive learning and attention mechanisms, respectively, to enhance clustering performance. In contrast, graph-level clustering approaches aim to embed entire graphs into a latent space and divide those exhibiting similar global features. For instance, DGLC [4] and DCGLC [6] utilize mutual information maximization to improve graph-level performance.

Although these two types of methods can achieve each other's functions with some adaptation (see Fig. 1), they essentially treat node-level and graph-level clustering as isolated tasks, which limits

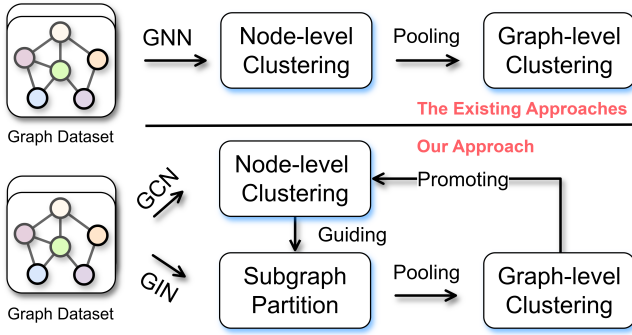


Figure 1: Compared to the existing graph clustering methods, our approach fully leverages the strengths of both levels to mutually enhance each other, rather than simply pooling node embedding to obtain graph-level features.

their ability to fully capture the spectrum of graph information [21]. Node-level clustering is effective in capturing local structures and fine-grained relationships, yet it remains limited in recognizing global patterns. In contrast, graph-level clustering focuses on global knowledge, aggregating information from the entire graph, yet may miss crucial details at the node level. This separation often results in suboptimal performance in tasks where both local and global information is essential. In drug discovery [27], molecules are represented as graphs, where nodes correspond to atoms and edges to chemical bonds. Effective clustering of drug molecules requires capturing both the local structural properties of individual atoms (node-level) and the overall molecular configuration (graph-level). A unified framework that integrates these two types of clustering can simultaneously leverage this multi-level information, enhancing the overall clustering process. Similarly, in **web-based social network analysis**, graph-level clustering and node-level clustering operate concurrently, mutually enhancing each other. While graph-level clustering groups users into broader communities, providing a higher-level organizational structure, node-level clustering can then fine-tune the distinction of individual users within those communities. Conversely, node-level clustering refines the structure of communities formed at the graph level by considering specific behaviors or attributes (e.g., interests, activity levels), ensuring that the resulting communities are more homogenous and meaningful.

Based on the analysis above, an intuitive approach is to fully integrate the information from both node-level and graph-level clustering, enabling them to optimize each other and complement one another. However, there are two challenges that need to be addressed. 1) We need to uncover the intrinsic relationships between the two modalities at a finer-grained pattern and extract guiding signals that are beneficial for each. This requires us to identify potential shared information between different modalities. 2) We need to design a learning paradigm that leverages prior advantage from both perspectives to promote their mutual enhancement. This process involves fully exploiting the complementary knowledge between the two perspectives while addressing potential conflicts.

To this end, we propose a novel framework called the Unified Graph Clustering Network (UGCN), which simultaneously performs node-level and graph-level clustering within a unified framework. Specifically, we employ a projector based on Graph Convolutional Networks (GCNs) to extract node-level embeddings and map them to distinct cluster layers, which are treated as prototypes. These prototypes, along with the distances from samples to them, are then used as priors to enhance the origin graph attributes, facilitating the separation of graphs into distinct substructures, which solves the first issue. Next, we apply Isomorphic Networks (GIN) to learn subgraph embeddings. To further refine the representation, we introduce joint triple-contrastive learning to maximize the mutual information between subgraphs, nodes, and node-subgraph pairs. This achieves knowledge complementarity from different levels, solving the second issue. Finally, we perform graph-level clustering, and the generated pseudo-labels are used to guide node-level clustering through K -means. In this approach, node-level prototypes serve as prior knowledge to guide graph-level clustering. Conversely, the pseudo labels generated from graph-level clustering provide the global context that refines the node-level clustering process to achieve the collaborative optimization. In summary, our contributions are summarized as follows:

- **New Perspective** To the best of our knowledge, this is the first attempt to unify node-level and graph-level tasks within a single framework, which presents a greater challenge compared to monolithic approaches.
- **New Task** We propose a unified framework to simultaneously clustering instances at both the graph and node levels. This framework achieves dual task collaborative optimization via prototype-guided joint triple contrastive learning.
- **Excellent Performances** Experimental results on two types of tasks on seven datasets show that our method significantly outperforms existing state-of-the-art methods.

2 Related Work

Related work is shown in **Appendix A**.

3 Methodology

In this section, we provide a detailed overview of UGCN. The central idea behind UGCN is to effectively harness both node-level local knowledge and graph-level global knowledge, enabling them to mutually enhance each other and improve overall clustering performance. In detail, we design a two-branch network to learn node-level embeddings and subgraph-level embeddings, respectively, and then optimize them by joint triple contrastive learning. The architecture of UGCN is illustrated in Fig. 2. The key steps involved in the UGCN framework are described in Algorithm 1.

3.1 Notations

Suppose that we have a graph dataset $\mathbb{G}$ with M graphs, denoted as $\mathbb{G} = \{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_M\}$. Each graph can be expressed as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is node set and $\mathcal{E}$ is edge set. Each graph have its own set of node features $\mathbf{X} \in \mathbb{R}^{n \times d}$ and adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ where d is the feature dimension and n is the number of nodes. Unless otherwise specified, graphs are assumed to be undirected, in which case $\mathbf{A}$ is symmetric. The entire dataset can be partitioned into N^ψ

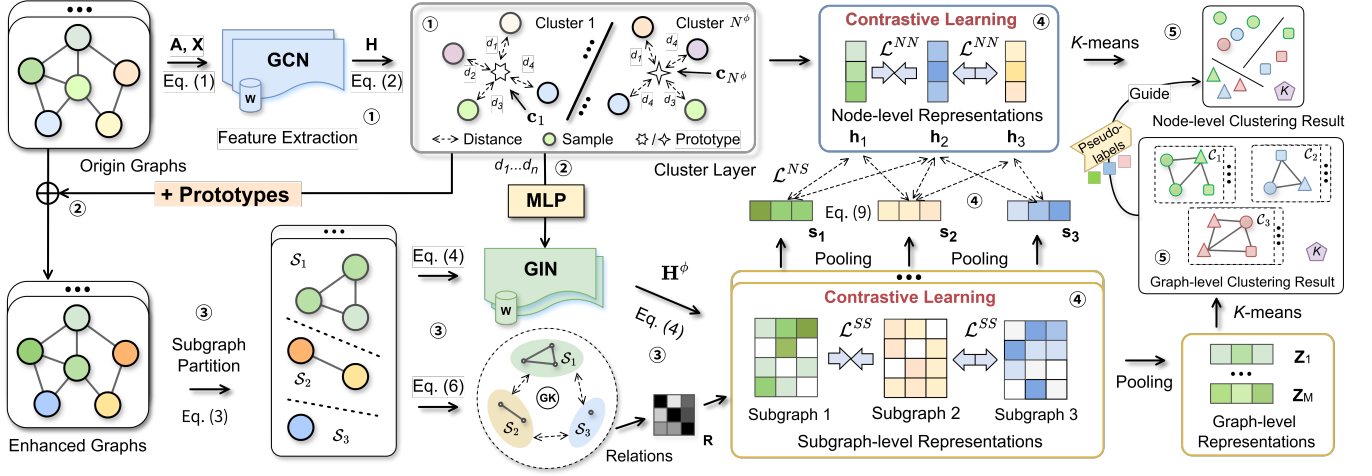


Figure 2: The Overall architecture of the proposed UGCN. ① The UGCN framework first employs the encoder in the first branch to extract node representations and learn prototypes. ② These prototypes, along with the distances between the prototypes and the samples, are subsequently used as prior knowledge to guide the graph-level learning process. ③ In parallel, the second branch partitions the graph structure into finer-grained subgraphs, from which subgraph representations are extracted. ④ Ultimately, both tasks are jointly optimized through a triple contrastive learning approach. ⑤ Additionally, in turn, the graph-level pseudo-labels are used to guide node-level graph clustering.

clusters at the graph level, while at the node level, it can be divided into N^ϕ clusters. The goal is to perform node-level graph-level clustering across a series of graphs.

3.2 Node-level Feature Learning

In this section, we use a projector to extract informative node-level features and derive prototypes utilizing cluster-based representations. These prototypes act as a critical guiding signal, facilitating the progressive refinement of the graph-level representation. Specifically, we first employ the random walk encoding [40, 41] X^ϕ to enhance the original node attribute for each node, as $X \leftarrow \text{CONCAT}(X, X^\phi)$, where $\text{CONCAT}(\cdot)$ is a connection function. Then, we design the dual-branch network to extract low-dimensional node features that encapsulate both the structural and semantic characteristics of the individual nodes within the graph. Here, the first branch of the network consists of the GCN layer [52], and the transformation at layer l can be expressed as:

$$\mathbf{h}_i^{(l)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \mathbf{W}^{(l)} \mathbf{h}_j^{(l-1)} + \mathbf{b}^{(l)} \right), \quad (1)$$

where $\mathcal{N}(i)$ represents the set of neighbors of node i , $\mathbf{W}^{(l)}$ is the learnable weight matrix at layer l , $\mathbf{h}_i^{(l)}$ denotes the latent representation of node i of l -level layer. and $\sigma(\cdot)$ is an activation function such as ReLU. This process aggregates local neighborhood information, enabling the model to learn meaningful node-level embeddings that form the basis. It is important to highlight that, inspired by SDCN [1], to fully obtain the key strengths of the autoencoder, we alternate between using the adjacency matrix A in half of the epochs and substituting it with the identity matrix I in the remaining epochs. In this way, the learned representations combine the unique advantages of both AE and GCN, while maintaining a simple and reliable network structure.

3.2.1 Centroid Layer as Prototypes. To generate clustering-friendly guidance signals that enhance graph-level properties, we introduce a centroid layer that learns prototypes, represented as $\mathbf{c}_k \in \mathbb{R}^d$ for $k \in \{1, 2, \dots, N^\phi\}$. The centroids are initialized and refined through the prior clustering task, with each centroid $\mathbf{c}_k$ acting as a representative prototype for the k -th cluster. The assignment of each node i to a centroid is expressed as:

$$\gamma_{ik} = \exp \left(- \frac{\|\mathbf{h}_i^{(L)} - \mathbf{c}_k\|^2}{\tau_k^2} \right) / \sum_{k'=1}^K \exp \left(- \frac{\|\mathbf{h}_i^{(L)} - \mathbf{c}_{k'}\|^2}{\tau_{k'}^2} \right), \quad (2)$$

where γ_{ik} represents the probability that node i belongs to cluster k , and τ_k^2 is a scaling factor that controls the softness of assignments. This responsibility function allows for soft clustering, where nodes are assigned to centroids probabilistically based on feature similarity. The centroids are continuously refined throughout the optimization process, ensuring that they accurately represent the core characteristics of the clusters they correspond to. Subsequently, we incorporate the distance differences between the prototypes and samples, relative to the prototypes, into both the node attributes and latent representations of the original graph. The distance differences are mapped through a Multi-Layer Perceptron (MLP). By doing so, we obtain cluster-level guidance signals, which guide the graph partitioning and representation learning processes.

3.3 Graph-level Feature Learning

After learning the node-level embeddings, we combine the raw attributes with prior knowledge to partition the graph into subgraphs, thereby driving the extraction of subgraph-level representations. The primary goal of this partitioning is to capture graph-level features at a finer granularity, ensuring that the inherent structural

relationships of the original graph are preserved while enhancing the cluster expressiveness of the learned representations.

3.3.1 Subgraph Feature Learning. Specifically, we first enhance the original node features with prototypes, as: $\mathbf{x}_i \leftarrow \text{CONCAT}(\mathbf{x}_i, \mathbf{c}_i)$ for each node i . In order to capture the fundamental patterns of the graph at a finer granularity, enabling mutual enhancement between them, high-quality graph partitioning is crucial for contrastive learning. The goal is to partition the graph into subgraphs such that nodes within the same subgraph are tightly clustered based on both semantic and structural similarities, while nodes across subgraphs are dissimilar. This partitioning approach aims to uncover more cohesive graph patterns. Specifically, we design the dynamic partitioning to minimize the normalized cut between subgraphs, which facilitates flexible partitioning by accounting for both graph structure and node attributes. By minimizing the dissimilarity between subgraphs, this approach leads to tighter and more informative subgraph representations. The normalized cut is defined as:

$$\text{Ncut}(\mathcal{S}_1, \mathcal{S}_2) = \frac{\sum_{i \in \mathcal{S}_1, j \in \mathcal{S}_2} w_{ij}}{\sum_{i \in \mathcal{S}_1} \sum_j w_{ij}} + \frac{\sum_{i \in \mathcal{S}_2, j \in \mathcal{S}_1} w_{ij}}{\sum_{j \in \mathcal{S}_2} \sum_i w_{ij}}, \quad (3)$$

where $w_{ij} = \text{sim}(\mathbf{x}_i, \mathbf{x}_j)$ is the weight of the edge between node i and node j , and $\mathcal{S}_1, \mathcal{S}_2$ are two partitions of the graph. The objective is to minimize this cut, ensuring that nodes within a partition are more strongly connected than nodes across partitions. To automatically identify potential substructures in the graph, we propose an adaptive partitioning algorithm, which recursively refines the graph partitioning process. At each stage of partitioning, the algorithm evaluates the quality of subgraphs to decide whether further partitioning is required, which is formally defined as: $Q(\mathcal{S}) = \frac{1}{2m} \sum_{i,j} \left(w_{ij} - \frac{d_i d_j}{2m} \right) \delta(\mathcal{S}_i, \mathcal{S}_j)$ where m is the total edge weight of the graph, d_i is the degrees of nodes i , and $\delta(\mathcal{S}_i, \mathcal{S}_j)$ is the indicator function, which equals 1 if nodes i and j belong to the same subgraph and 0 otherwise. A higher value indicates that a subgraph is more tightly connected internally while having fewer connections to other subgraphs, making it a potential substructure. To quantify the density of a subgraph, we calculate it as: $\text{Density}(\mathcal{S}) = \frac{\sum_{i \in \mathcal{S}, j \in \mathcal{S}} w_{ij}}{\text{Si}(\mathcal{S})}$ where $\text{Si}(\mathcal{S})$ is the number of nodes in subgraph $\mathcal{S}$. During each partitioning step, we use Eq. (3) to minimize the dissimilarity between subgraphs.

3.3.2 Graph-Level Representation Learning. Once the graph has been partitioned into subgraphs using one of the advanced methods above, we proceed with learning node embeddings using Graph Isomorphism Networks (GIN). GIN is particularly powerful because it can capture rich local structural information in a graph, making it ideal for learning discriminative node representations in subgraphs. The update rule for GIN is given by:

$$\mathbf{h}_i^{(l+1)} = \text{MLP}\left(\mathbf{h}_i^{(l)} + \epsilon^{(l)} \sum_{j \in \mathcal{N}(i)} \mathbf{h}_j^{(l)}\right), \quad (4)$$

where $\mathbf{h}_i^{(l)}$ is the feature vector of node i at layer l , $\mathcal{N}(i)$ is the set of neighbors of node i , $\text{MLP}(\cdot)$ is a multi-layer perceptron used to perform a non-linear transformation. We apply GIN on each subgraph $\mathcal{S}_i$ separately to learn rich node representations $\mathbf{H}_i^\phi$, which capture the structural and semantic properties of the graph. Pooling $\mathbf{H}_i^\phi$ can

Algorithm 1: Training Procedure of UGCN

Input: Training graph $\mathbb{G} = \{G_i\}_{i=1}^N$ represented as $(\mathbf{X}, \mathbf{A})$;
 Training Epochs N_{epoch} ;
Output: Clustering Results R .

- 1 **for** $e \leftarrow 0$ **to** N_{epoch} **do**
- 2 Learn the node embeddings $\mathbf{H}$ by Eq. (1);
- 3 Obtain prototypes $\mathbf{C}$ by Eq. (2);
- 4 Enhance graph-level origin attribute with prototypes;
- 5 Partition the original graph into subgraphs by Eq. (3);
- 6 Learn the subgraph embeddings $\mathbf{S}$ by Eq. (4);
- 7 Calculate node-to-node contrastive loss $\mathcal{L}_{\text{con}}^{NN}$ by Eq. (5);
- 8 Utilize the graph kernel method to rebuild the relation between multiple subgraphs by Eq. (6).
- 9 Utilize the relation of subgraphs to calculate subgraph-to-subgraph contrastive loss $\mathcal{L}_{\text{con}}^{SS}$ by Eq. (7);
- 10 Utilize node weights calculate node-to-subgraph contrastive loss $\mathcal{L}_{\text{con}}^{NS}$ by Eq. (9);
- 11 Backpropagation for parameter updates;
- 12 **end**
- 13 Perform graph-level clustering by K -means;
- 14 Perform K -means node-level clustering guided by graph-level pseudo-labels;
- 15 **return** R ;

obtain the subgraph-level representation $\mathbf{S}$ of the subgraph. During this learning process, the distance between nodes and prototypes is mapped through an MLP and subsequently concatenated with the latent representations produced by GIN, with the aim of enhancing the model's ability to incorporate prototype-guided information and refine the node embeddings.

3.4 Triple Joint Contrastive Optimization

After obtaining the node embeddings using GIN and GCN. We apply contrastive learning to further optimize the sample representations. Here, we define three optimization objectives on different levels: node-to-node, subgraph-to-node, and subgraph-to-subgraph levels. The objective of contrastive learning is to maximize the similarity of the same pattern subgraph while minimizing the similarity between different samples.

3.4.1 Node-to-Node Contrastive. For node-to-node contrastive learning in the dual network, the loss is defined as:

$$\mathcal{L}_{\text{con}}^{NN} = - \sum_{i=1}^n \log \frac{\sum_{j \in \mathcal{N}^+(i)} \exp(\mathbf{h}_i \cdot \mathbf{h}_j / \tau)}{\sum_{j=1}^n \exp(\mathbf{h}_i \cdot \mathbf{h}_j / \tau)} + \|\mathbf{h}_i - \mathbf{h}_i^\phi\|, \quad (5)$$

where $\mathcal{N}^+(i)$ refers to a positive sample derived from the neighbors of node i . τ is the temperature parameter controlling the distribution sharpness. Here, the first term aims to bring adjacent positive samples closer within the unified network, while the second term focuses on bringing different representations of the same node across different networks closer together.

3.4.2 Subgraph-to-Subgraph Contrastive. For subgraph-to-subgraph contrastive loss, we reconstruct the relation by the Graph Kernel (GK) method (e.g., SP, LT Kernel), as $r_{ij} = S(G_i, G_j)$ where $S(G_i, G_j)$

denote the graph kernel similarity between graphs G_i and G_j . The classification of pairs as positive or negative samples can be expressed as:

$$r_{ij} = \begin{cases} 1 & \text{if } S(G_i, G_j) > T \\ 0 & \text{if } S(G_i, G_j) \leq T \end{cases}, \quad (6)$$

where T denotes a threshold hyper-parameter. $r_{ij} = 1$ indicates a positive sample (high similarity), $r_{ij} = 0$ indicates a negative sample (low similarity). This formula classifies pairs of graphs as positive if their similarity exceeds half of the maximum similarity, and negative otherwise. By doing so, we reconstruct the relation matrix $\mathbf{R}$ from various subgraphs. Next, we can use these associations to optimize the representation of similar subgraphs while distinguishing dissimilar subgraphs. The subgraph-to-subgraph contrastive loss can be expressed as follows

$$\mathcal{L}_{\text{con}}^{\text{SS}} = - \sum_{i,j} r_{ij} \log \frac{\exp(\mathbf{h}_i^{\text{SS}} \cdot \mathbf{h}_j^{\text{SS}} / \tau)}{\sum_{k=1}^n \exp(\mathbf{h}_i^{\text{SS}} \cdot \mathbf{h}_k^{\text{SS}} / \tau)}. \quad (7)$$

3.4.3 Node-to-Subgraph Contrastive. Furthermore, to fully leverage the complementary features of the two networks, we design the node-to-subgraph loss to further enhance the overall representation. This step ensures that the learned node representations not only align with the global graph structure but also adapt to localized patterns within subgraphs. Specifically, we first combine local and global structure information to measure node importance. The local structural weight $C_L(v_i)$ of node i is expressed as: $C_L(i) = d_i \cdot \sum_{j \in \mathcal{N}(i)} \frac{1}{d_j}$. A higher weight indicates greater importance in the local structure. The global structural importance $C_B(i)$ of the node i is defined as: $C_B(i) = \sum_{s \neq i \neq t} \frac{\sigma(s, t|i)}{\sigma(s, t)}$ where $\sigma(s, t)$ is the number of shortest paths from node s to node t , and $\sigma(s, t|i)$ is the number of shortest paths through node i . Moreover, we introduce the eigenvector centrality $C_E(i)$ of a node i . Finally, the total importance weight ξ_i of a node i is given by the following formula:

$$\xi_i = \mu \cdot C_L(i) + (1 - \mu) \cdot (C_B(i) + C_E(v_i)), \quad (8)$$

where $\mu \in [0, 1]$ is a hyperparameter that balances the contributions of local and global structure, which is empirically set to 0.5. In this way, we can more subtly reflect the potential connections between nodes and subgraphs. The subgraph-to-node contrastive loss is designed to maximize the mutual information between subgraph-level embeddings and the nodes that form those subgraphs, as:

$$\mathcal{L}_{\text{con}}^{\text{NS}} = - \sum_{i \in S_k} \xi_i \left[\log \sigma(T(\mathbf{s}_k, \mathbf{z}_i)) + \log(1 - \sigma(T(\mathbf{s}_k, \tilde{\mathbf{z}}))) \right], \quad (9)$$

where S_k is a set that contains all nodes on subgraph k , $\tilde{\mathbf{z}}$ is the negative samples, and T is the mutual information discriminator. This loss function ensures that the model learns to separate nodes from different subgraphs while bringing nodes from the same subgraph closer together in the embedding space. The final loss function combines the contrastive losses at all three levels to the following overall optimization objective:

$$\mathcal{L}_{\text{total}} = \alpha_1 \times \mathcal{L}_{\text{con}}^{\text{NN}} + \alpha_2 \times \mathcal{L}_{\text{con}}^{\text{NS}} + \alpha_3 \times \mathcal{L}_{\text{con}}^{\text{SS}}. \quad (10)$$

The integration of these three contrastive learning objectives ensures that the model learns to capture both local and global graph features, and it enhances the discriminative power of the embeddings. The final representation is thus refined through the interplay

Table 1: Statistics of the multiple graph datasets. A.Nodes and A.Edges denote average node numbers and average edges, respectively. “-” denotes that the node attributes do not exist.

Datasets	Graphs	A.Nodes	A.Edges	N.Attrs	N.Classes	G.Classes
COX2	467	41.22	86.89	+(3)	8	2
BZR	405	35.75	76.72	+(3)	10	2
DHFR	756	42.43	89.09	+(3)	17	2
PROTEINS	1113	39.06	145.63	+(1)	3	2
ENZYMES	600	32.63	124.27	+(18)	3	6
SW-620	40532	26.06	56.17	-	65	2
PC-3	27509	26.36	56.98	-	45	2

between these levels, improving the model’s ability to distinguish between different graph structures and their underlying patterns.

Finally, the learned graph-level representations are input into K -means for graph-level clustering, generating pseudo-labels. These pseudo-labels are then integrated with the node-level features to guide node-level clustering. The entire training process establishes a closed-loop in which node-level clustering provides prior information to facilitate graph-level clustering, while graph-level clustering knowledge in turn influences node-level clustering. This iterative feedback mechanism allows the model to jointly optimize both tasks, thereby enhancing the performance of each task in a complementary manner.

3.5 Complexity Analysis

The Computational complexity of the UGCN approach can be broken down into several stages, each contributing differently based on the operations involved. In the encoding stage, L -layer GIN and GCN networks require $O(|\mathcal{E}|dL + nd^2L)$ time complexity. Partitioning each graph into subgraphs by N-cut requires $O(n^2d)$ complexity. The triple constraining requires $O(n^2d)$ complexity. Overall Complexity: Overall, the total time complexity of the UGCN approach is dominated by the combination of these stages: $O(|\mathcal{E}|dL + nd^2L + n^2d + n^2d)$. This reflects the overall computational cost across the encoding, partitioning, and constraining steps.

4 Experiments

In this section, we provide a thorough evaluation of UGCN. We begin with a series of comparison experiments against SOTA baselines, followed by ablation studies to assess the contribution of each individual component. We also conduct a hyperparameter sensitivity analysis to examine the robustness of our approach across different configurations. Additionally, we perform convergence experiments to verify the stability and effectiveness of UGCN.

4.1 Experiential Settings

4.1.1 Evaluation Metrics. We adopt Accuracy (ACC) [29, 43], Normalized Mutual Information (NMI) [22, 25], Adjusted Rand Index (ARI) [24], and F1 [28] score as the primary evaluation metrics, which are widely regarded as standard in clustering evaluation [26, 33]. The details of metrics are provided in **Appendix B**.

4.1.2 Implementation Details. For a fair comparison, each experiment is independently repeated five times. We use the Adam optimizer with a learning rate of 0.001, which decays at a rate of $5e - 4$,

Table 2: Performance comparison results (expressed as mean $\pm$ standard deviation) of the node-level and graph-level clustering task on five datasets. Bold and underlined figures represent the highest and second-highest performances for any column, respectively. “OOM” represents memory limit exceeded.

Methods	Node-level Clustering																			
	COX2				BZR				DHFR				PROTEINS				SW-620			
	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1
AE	51.3 $\pm$ 0.0	0.1 $\pm$ 0.0	0.0 $\pm$ 0.0	24.4 $\pm$ 1.2	26.2 $\pm$ 0.3	3.5 $\pm$ 0.4	1.6 $\pm$ 0.2	11.3 $\pm$ 0.4	42.6 $\pm$ 0.1	0.1 $\pm$ 0.0	0.0 $\pm$ 0.0	6.6 $\pm$ 0.0	68.7 $\pm$ 0.3	16.9 $\pm$ 0.5	14.2 $\pm$ 0.3	45.2 $\pm$ 0.4	73.7 $\pm$ 0.2	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.1 $\pm$ 0.0
GAE	23.8 $\pm$ 0.2	3.6 $\pm$ 0.3	2.5 $\pm$ 1.9	12.7 $\pm$ 1.6	51.2 $\pm$ 0.0	1.6 $\pm$ 0.0	0.1 $\pm$ 0.0	23.8 $\pm$ 0.0	40.5 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	7.8 $\pm$ 0.2	67.3 $\pm$ 0.3	14.2 $\pm$ 0.1	12.5 $\pm$ 0.2	38.7 $\pm$ 0.2	64.3 $\pm$ 0.2	0.0 $\pm$ 0.0	0.1 $\pm$ 0.0	0.2 $\pm$ 0.0
DEC	54.7 $\pm$ 0.0	4.7 $\pm$ 0.1	0.9 $\pm$ 0.0	36.5 $\pm$ 1.2	42.3 $\pm$ 0.5	8.6 $\pm$ 0.5	7.1 $\pm$ 0.6	15.9 $\pm$ 0.4	37.6 $\pm$ 0.2	0.8 $\pm$ 0.1	0.4 $\pm$ 0.2	10.3 $\pm$ 1.1	60.8 $\pm$ 0.3	20.3 $\pm$ 0.5	16.7 $\pm$ 0.3	48.6 $\pm$ 0.4	72.4 $\pm$ 0.1	1.1 $\pm$ 0.2	1.2 $\pm$ 0.1	1.5 $\pm$ 0.1
SDCN	57.6 $\pm$ 0.0	5.1 $\pm$ 0.2	2.7 $\pm$ 0.0	38.8 $\pm$ 0.9	61.5 $\pm$ 0.8	15.7 $\pm$ 0.4	12.6 $\pm$ 1.8	43.5 $\pm$ 0.6	58.2 $\pm$ 0.6	3.5 $\pm$ 0.3	2.8 $\pm$ 1.5	19.5 $\pm$ 1.3	72.4 $\pm$ 0.5	22.3 $\pm$ 0.3	25.6 $\pm$ 0.6	42.3 $\pm$ 0.4	75.6 $\pm$ 0.5	2.3 $\pm$ 0.1	2.2 $\pm$ 0.2	3.8 $\pm$ 0.2
DFCN	50.4 $\pm$ 0.0	0.2 $\pm$ 0.0	0.0 $\pm$ 0.0	32.3 $\pm$ 2.1	55.4 $\pm$ 0.2	10.3 $\pm$ 1.1	8.7 $\pm$ 0.5	26.5 $\pm$ 0.3	46.4 $\pm$ 2.0	2.1 $\pm$ 0.8	2.0 $\pm$ 0.2	12.7 $\pm$ 1.6	63.8 $\pm$ 0.3	17.6 $\pm$ 0.2	15.7 $\pm$ 1.5	36.6 $\pm$ 0.3	69.2 $\pm$ 0.1	0.5 $\pm$ 0.1	0.7 $\pm$ 0.2	2.6 $\pm$ 0.9
DCLN	56.9 $\pm$ 2.6	0.6 $\pm$ 1.0	2.1 $\pm$ 1.3	40.8 $\pm$ 1.0	52.7 $\pm$ 0.8	3.3 $\pm$ 1.1	0.0 $\pm$ 0.0	26.4 $\pm$ 0.9	57.3 $\pm$ 2.1	6.2 $\pm$ 0.8	4.7 $\pm$ 0.6	20.4 $\pm$ 0.5	55.8 $\pm$ 1.3	12.3 $\pm$ 0.5	13.2 $\pm$ 0.4	28.3 $\pm$ 0.7	64.5 $\pm$ 0.7	5.2 $\pm$ 0.2	4.4 $\pm$ 0.3	5.9 $\pm$ 0.5
SCGC	<u>66.5$\pm$0.5</u>	<u>8.1$\pm$0.3</u>	<u>10.6$\pm$0.4</u>	<u>52.9$\pm$1.2</u>	63.6 $\pm$ 0.0	1.1 $\pm$ 0.0	1.1 $\pm$ 3.4	42.6 $\pm$ 2.5	<u>66.8$\pm$0.2</u>	<u>5.7$\pm$1.3</u>	<u>12.9$\pm$0.6</u>	<u>42.4$\pm$0.5</u>	68.3 $\pm$ 1.4	23.5 $\pm$ 0.2	26.7 $\pm$ 1.1	46.5 $\pm$ 0.3	63.8 $\pm$ 1.1	<u>17.6$\pm$0.4</u>	<u>19.5$\pm$1.2</u>	<u>51.6$\pm$1.4</u>
ABR	64.3 $\pm$ 0.6	7.9 $\pm$ 0.4	3.5 $\pm$ 0.1	42.4 $\pm$ 0.2	<u>66.8$\pm$0.2</u>	<u>18.9$\pm$0.3</u>	<u>14.2$\pm$0.2</u>	<u>58.3$\pm$0.7</u>	61.5 $\pm$ 0.4	4.3 $\pm$ 0.1	3.4 $\pm$ 0.7	23.8 $\pm$ 0.1	<u>73.5$\pm$0.3</u>	<u>23.8$\pm$0.2</u>	<u>27.5$\pm$0.4</u>	<u>50.1$\pm$0.7</u>	<u>76.2$\pm$0.3</u>	4.3 $\pm$ 0.7	3.7 $\pm$ 0.2	10.3 $\pm$ 0.5
OURS	78.5$\pm$0.2	15.3$\pm$0.4	20.3$\pm$0.4	57.6$\pm$1.3	80.1$\pm$1.3	21.6$\pm$0.4	25.2$\pm$1.4	72.3$\pm$1.5	71.4$\pm$0.6	8.9$\pm$0.1	15.6$\pm$0.3	60.5$\pm$1.0	74.6$\pm$0.9	27.3$\pm$0.4	31.3$\pm$0.6	55.3$\pm$0.4	78.1$\pm$0.4	20.6$\pm$0.2	21.3$\pm$0.1	60.2$\pm$0.5
	Graph-level Clustering																			
	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1
	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1
RW	51.3 $\pm$ 0.0	0.7 $\pm$ 0.0	0.0 $\pm$ 0.0	37.5 $\pm$ 0.0	60.6 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	24.6 $\pm$ 0.0	51.3 $\pm$ 0.0	0.1 $\pm$ 0.0	0.0 $\pm$ 0.0	24.4 $\pm$ 1.2	64.4 $\pm$ 0.0	6.0 $\pm$ 0.0	5.9 $\pm$ 0.0	45.5 $\pm$ 0.0	61.3 $\pm$ 0.0	3.1 $\pm$ 0.0	2.4 $\pm$ 0.0	28.5 $\pm$ 0.0
LT	77.5 $\pm$ 0.6	0.2 $\pm$ 0.0	0.1 $\pm$ 0.0	55.5 $\pm$ 0.8	78.3 $\pm$ 0.4	0.7 $\pm$ 0.3	1.1 $\pm$ 1.0	26.9 $\pm$ 0.3	51.2 $\pm$ 0.0	1.6 $\pm$ 0.0	0.1 $\pm$ 0.0	23.8 $\pm$ 0.0	60.3 $\pm$ 0.0	1.5 $\pm$ 0.0	0.8 $\pm$ 0.0	44.4 $\pm$ 0.0	63.6 $\pm$ 0.0	4.7 $\pm$ 0.0	3.5 $\pm$ 0.0	32.4 $\pm$ 0.0
InfoGraph	56.4 $\pm$ 3.2	3.3 $\pm$ 0.6	0.2 $\pm$ 0.0	37.1 $\pm$ 2.6	63.6 $\pm$ 0.4	1.6 $\pm$ 0.3	2.4 $\pm$ 1.0	33.4 $\pm$ 0.2	54.7 $\pm$ 0.0	4.7 $\pm$ 0.1	0.9 $\pm$ 0.0	36.5 $\pm$ 1.2	59.2 $\pm$ 0.2	3.2 $\pm$ 1.9	0.0 $\pm$ 0.0	52.6 $\pm$ 0.3	50.9 $\pm$ 0.1	0.2 $\pm$ 0.1	0.2 $\pm$ 0.1	22.4 $\pm$ 0.2
GraphCL	68.8 $\pm$ 0.6	1.0 $\pm$ 0.2	0.4 $\pm$ 0.5	44.2 $\pm$ 1.8	72.8 $\pm$ 4.0	1.0 $\pm$ 0.8	3.4 $\pm$ 1.0	38.1 $\pm$ 0.7	57.6 $\pm$ 0.0	5.1 $\pm$ 0.2	2.7 $\pm$ 0.0	38.8 $\pm$ 0.9	59.4 $\pm$ 0.0	2.8 $\pm$ 0.1	3.8 $\pm$ 0.1	47.6 $\pm$ 0.1	51.2 $\pm$ 0.0	0.5 $\pm$ 0.2	0.3 $\pm$ 0.2	29.7 $\pm$ 0.3
JOAO	70.5 $\pm$ 2.0	1.2 $\pm$ 0.3	0.5 $\pm$ 0.4	38.4 $\pm$ 0.9	72.6 $\pm$ 0.0	2.7 $\pm$ 1.4	5.6 $\pm$ 3.7	42.1 $\pm$ 1.1	50.4 $\pm$ 0.0	0.2 $\pm$ 0.0	0.0 $\pm$ 0.0	32.3 $\pm$ 0.1	59.8 $\pm$ 0.0	0.4 $\pm$ 0.0	0.1 $\pm$ 0.0	43.5 $\pm$ 0.1	53.6 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	32.4 $\pm$ 0.1
GFW	57.6 $\pm$ 4.1	1.5 $\pm$ 0.1	2.1 $\pm$ 1.8	39.2 $\pm$ 0.0	52.7 $\pm$ 0.8	3.3 $\pm$ 1.1	0.0 $\pm$ 0.0	26.4 $\pm$ 0.9	56.9 $\pm$ 2.6	0.6 $\pm$ 1.0	2.1 $\pm$ 1.3	40.8 $\pm$ 1.0	66.8 $\pm$ 2.3	9.0 $\pm$ 1.2	11.4 $\pm$ 3.1	54.2 $\pm$ 2.9	56.7 $\pm$ 2.4	3.3 $\pm$ 1.9	3.0 $\pm$ 1.2	34.6 $\pm$ 1.5
GLCC	77.3 $\pm$ 0.0	0.1 $\pm$ 0.3	0.0 $\pm$ 0.0	42.1 $\pm$ 0.0	63.6 $\pm$ 0.0	1.1 $\pm$ 0.0	1.1 $\pm$ 3.4	42.6 $\pm$ 2.5	66.5 $\pm$ 0.0	8.1 $\pm$ 0.0	<u>10.6$\pm$0.0</u>	52.9 $\pm$ 1.2	60.6 $\pm$ 2.6	2.0 $\pm$ 1.4	4.16 $\pm$ 2.2	47.3 $\pm$ 2.1	57.2 $\pm$ 2.0	4.4 $\pm$ 1.3	3.2 $\pm$ 1.7	32.1 $\pm$ 1.8
UDGC	74.6 $\pm$ 1.8	0.5 $\pm$ 1.1	1.4 $\pm$ 2.2	42.2 $\pm$ 1.5	58.9 $\pm$ 2.1	0.7 $\pm$ 0.1	0.9 $\pm$ 0.0	33.3 $\pm$ 0.2	<u>68.7$\pm$0.2</u>	<u>10.4$\pm$1.3</u>	3.5 $\pm$ 2.3	46.9 $\pm$ 2.5	63.5 $\pm$ 1.4	1.8 $\pm$ 1.7	3.66 $\pm$ 1.3	51.3 $\pm$ 1.4	50.0 $\pm$ 1.2	0.0 $\pm$ 0.0	0.1 $\pm$ 0.0	24.6 $\pm$ 0.3
DGLC	78.2 $\pm$ 0.2	2.3 $\pm$ 0.1	6.7 $\pm$ 3.4	66.3 $\pm$ 2.1	80.9 $\pm$ 0.6	9.8 $\pm$ 0.9	20.5 $\pm$ 1.8	73.0 $\pm$ 2.4	66.2 $\pm$ 0.2	8.9 $\pm$ 0.5	4.4 $\pm$ 1.4	<u>53.6$\pm$3.3</u>	65.7 $\pm$ 0.4	7.2 $\pm$ 1.6	6.5 $\pm$ 0.9	51.1 $\pm$ 0.2	<u>64.3$\pm$1.2</u>	<u>5.4$\pm$1.5</u>	<u>4.7$\pm$1.1</u>	<u>35.8$\pm$0.6</u>
DCGLC	<u>78.3$\pm$1.4</u>	<u>4.3$\pm$0.6</u>	<u>7.8$\pm$1.5</u>	<u>68.5$\pm$1.1</u>	<u>81.1$\pm$0.2</u>	<u>13.7$\pm$1.8</u>	<u>18.6$\pm$1.2</u>	<u>74.8$\pm$2.3</u>	64.3 $\pm$ 0.3	7.8 $\pm$ 0.5	3.7 $\pm$ 1.2	52.5 $\pm$ 2.1	<u>68.8$\pm$2.0</u>	<u>10.9$\pm$1.35</u>	<u>14.3$\pm$2.8</u>	<u>60.4$\pm$3.7</u>	OOM	OOM	OOM	OOM
OURS	83.5$\pm$0.4	12.4$\pm$0.6	8.7$\pm$0.5	76.2$\pm$1.3	85.6$\pm$1.6	16.5$\pm$0.2	23.1$\pm$1.6	77.9$\pm$1.0	69.2$\pm$0.8	11.0$\pm$0.3	13.5$\pm$0.4	56.4$\pm$0.9	71.5$\pm$1.4	13.4$\pm$1.5	17.2$\pm$1.6	63.8$\pm$1.4	68.7$\pm$1.2	6.9$\pm$1.4	5.8$\pm$1.3	40.5$\pm$1.6



Figure 3: Clustering performance under different variants of UGCN on five datasets, reporting ACC, NMI, ARI, and F1 metrics.

and a batch size of 256 on all datasets. For additional implementation details of UGCN, see **Appendix C**.

4.1.3 Benchmark Datasets. To assess the generalizability of UGCN, we perform node-level and graph-level clustering on seven publicly available datasets from the TUDataset collection, spanning various clustering numbers and domains, including COX2, BZR, DHFR, PROTEINS, and ENZYMES. Additionally, to ensure that UGCN maintains consistent performance on larger and imbalanced datasets, we select SW-620 and PC-3 as additions. The statistical information of the dataset is shown in Fig. 1. The detailed information about these datasets can be found in **Appendix D**.

4.1.4 Baseline Methods. To comprehensively evaluate UGCN, we compare it with baselines across node-, and graph-level clustering tasks. Each type of task is divided into classic methods, early deep methods, and advanced deep methods. For node-level task, classic methods include AE [18], GCN [44] and DEC [14], early deep methods: SDCN [1], DFCN [42], and DCLN [37], advanced methods include SCGC [20], ABR [29]. For graph-level tasks, classic methods include: RW Kernel [2], LT Kernel [16]. early deep methods includes: InfoGraph [38], GraphCL [58], JOAO [57], and GFW [55], advanced methods include: GLCC [17], UDGC [13], DGLC [6],

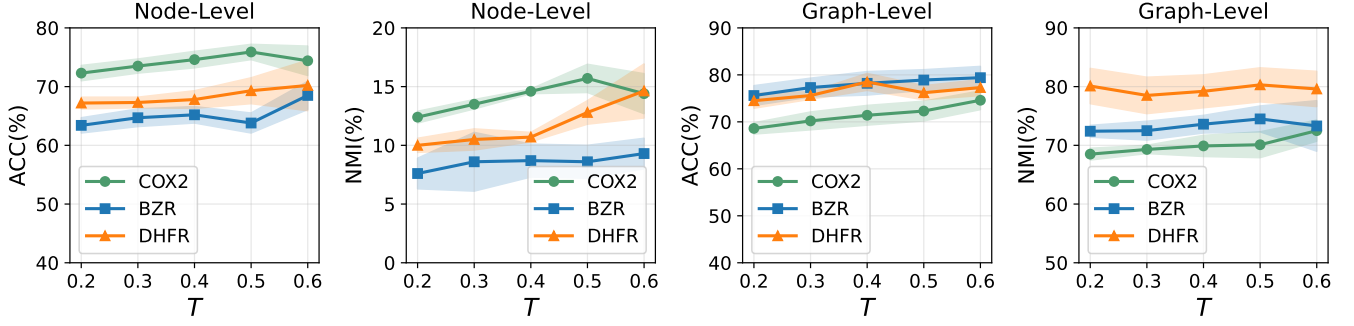


Figure 4: Sensitivity analysis of hyperparameters T on node-level and graph-level tasks, reporting ACC, NMI metrics.

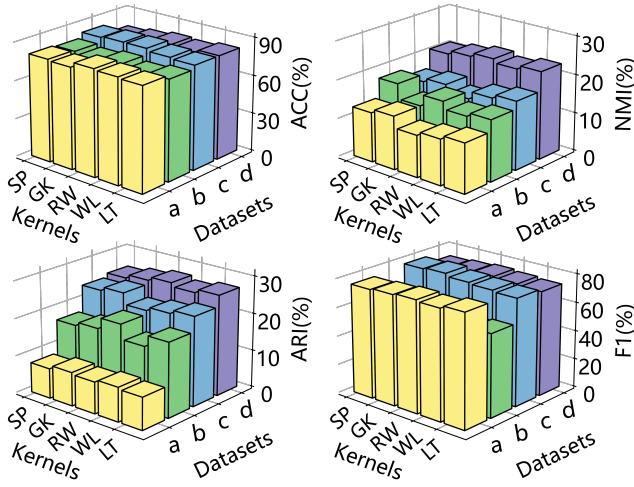


Figure 5: Clustering performance under different graph kernels. “a”, “b”, “c”, and “d” denote graph-level clustering on COX2, node-level clustering on COX2, graph-level clustering on BZR, node-level clustering on BZR, respectively.

DCGLC [5]. The details of these baseline methods are shown in Appendix E.

4.2 Comparison Experiments

To evaluate the effectiveness and performance of our proposed method. We compare it against several SOTA approaches across multiple benchmark datasets. The experimental results on five datasets are shown in Table 2, others are shown in Appendix F, and the following conclusions are drawn: 1) Compared to existing SOTA node-level clustering methods, our approach shows significant performance advantages. This is primarily because these methods typically focus on identifying patterns across different nodes within a single graph and fail to effectively leverage graph-level priors on multiple graphs to further refine the clustering boundaries. Alternatively, it can be argued that these methods lack the capability to effectively capture global knowledge. 2) Compared to advanced graph-level clustering methods, our approach demonstrates a significant performance advantage. This is primarily due

to the fact that they focus mainly on node- and graph-level features, without capturing the finer-grained for refining, intrinsic knowledge of the graph. Additionally, the absence of node-level prototype guidance in these methods leads to suboptimal performance. 3) Whether for node-level or graph-level tasks, traditional machine learning methods perform significantly worse than deep methods based on representation learning. At the same time, we also found that the fusion of multi-source information is beneficial for improving model performance. 4) In comparison, our method demonstrates a more balanced improvement in performance across node-level tasks. The significant improvements in NMI and ARI for graph-level tasks highlight the advantages of our approach, primarily due to the effective guidance from prototypes, which enhances the cohesion of graph-level samples.

4.3 Module Ablation Studies

In this section, we conduct ablation studies to assess the impact of each component in UGCN. We evaluate three model variants: 1) Removing the guidance from the prototype to the graph-level attributes (**w/o GP**), 2) Removing the subgraph partition, replaced by the entire graphs (**w/o SP**), and 3) Removing the joint triple contrastive loss (**w/o TC**), replaced by reconstruct loss. The experimental results, shown in Fig. 3, lead to the following conclusions: i) Removing the guidance from the prototypes results in a decline in model performance, mainly due to the absence of effective cluster signals, which makes accurate subgraph partitioning difficult and disrupts the mutual learning process of similar samples in contrastive learning. ii) Removing subgraph partitioning leads to a performance degradation of the model, primarily due to two factors: 1) the model fails to leverage subgraph-level contrastive learning for mutual enhancement, thereby hindering the extraction of more fundamental features; 2) due to the heterogeneous distribution patterns within a single graph, aligning the entire graph with the nodes disrupts personalized subgraph knowledge that is beneficial for clustering. 3) Removing triple contrastive learning and replacing it with reconstruction learning leads to a severe performance degradation, which is mainly attributed to the model inability to leverage the correlation between the two views to promote each other and learn a good representation of the clusters.

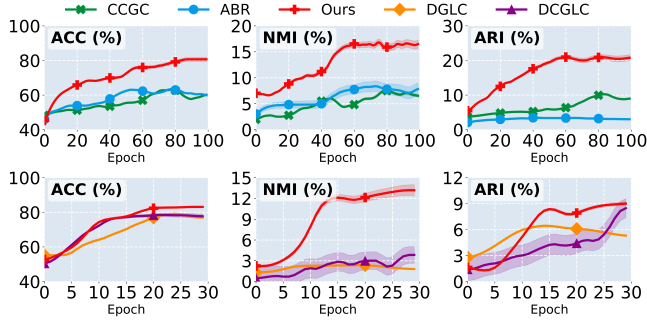


Figure 6: Convergence curve of the UGCN over 100 and 30 epochs for node-level (first line) and graph-level (second line) tasks on the COX2 dataset, respectively .

4.4 Graph Kernel Ablation Studies

In the methods section, a kernel is used to calculate the similarity between subgraphs. We perform ablation studies to evaluate the impact of different graph kernels on model performance. Specifically, we select the RW, SP, LT, and WL graph kernels. The experimental results are shown in Fig. 5, leading to the following conclusions: Reconstructing subgraph associations using different kernel methods introduces certain variations in model performance, which can primarily be attributed to the distinct metric patterns of each kernel. The SP and RW kernels emphasize structural similarity, capturing the topological relationships between subgraphs, while the LT and WL kernels focus more on attribute-based similarities, incorporating the features of nodes or subgraphs. Regardless of the chosen graph kernel, UGCN consistently demonstrates stable performance, reflecting its robustness across varying similarity metrics.

4.5 Hyperparameter Sensitivity Analysis

We perform a comprehensive sensitivity analysis to assess the impact of the key hyperparameters on the performance of UGCN.

4.5.1 Delimiting Parameter T . The parameter T controls the threshold for building similarity-based edges. A larger value of T results in fewer edges, while a smaller value leads to a denser graph. In this analysis, we vary T from 0.2 to 0.6 to assess its impact on model performance. The experimental results are shown in Fig. 4, and the following conclusion can be observed: 1) Choosing a value of T that is either too large or too small does not contribute to performance improvement; instead, selecting a moderate value for T is more appropriate. This is primarily because an excessive number of connections introduces noise, while too few connections cause subtle relationships to be overlooked, ultimately hindering the model’s ability to capture meaningful patterns in the data. 2) Compared with node-level clustering performance, graph-level clustering is more sensitive to changes in T . This can be primarily attributed to the fact that effective graph grouping depends on accurately capturing subgraph relationships, which enables the learning of more discriminative representations.

4.5.2 Loss Ratio α_1, α_2 , and α_3 . In order to explore the impact of different loss ratios on experimental performance, we set α_1, α_2 , and

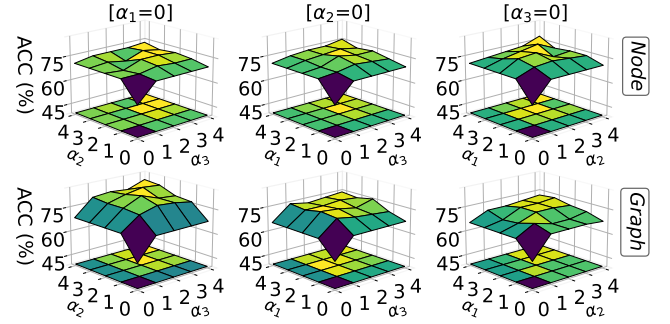


Figure 7: Clustering performance for node-level and graph-level tasks on COX2 dataset under various ratios of $\alpha_1, \alpha_2, \alpha_3$ from $[0:0.4:4]$, reporting ACC metric.

α_3 to 0 and dynamically adjusted the other two parameters to observe performance variations. The main results are presented in Fig. 7, while additional results are provided in Appendix F, which lead to the following conclusions. Node-level tasks are more sensitive to the node-to-node and node-to-subgraph contrastive losses, while graph-level tasks are most sensitive to the node-to-subgraph and subgraph-to-subgraph contrastive losses. Additionally, removing any of the loss components results in a noticeable performance decline, while the best performance is achieved only when all three losses are incorporated with balanced proportions.

4.6 Convergence Analysis

We analyze the convergence behavior of our model during training. By plotting the loss and evaluation metrics over time, we assess how quickly and steadily the model converges to an optimal solution. Analysis of the Fig. 6 reveals that, in comparison to existing methods, our approach achieves faster convergence while maintaining stability, thereby providing further evidence of the superiority of the proposed method. In addition, we found that graph-level tasks converge faster than node-level tasks.

5 Conclusion

In this paper, we propose a unified graph clustering network that collaboratively optimizes both node-level and graph-level tasks. It addresses the limitation of existing methods, which fail to establish relationships between tasks at different levels to enhance overall performance. The experimental results demonstrate that, whether at the node-level or graph-level, our proposed mechanism significantly enhances performance. This improved synergy between node and graph-level tasks leads to more precise and scalable clustering solutions, showcasing the potential of our unified approach in real-world applications. In the future, we plan to extend this approach to semi-supervised learning by incorporating a small amount of labeled data, which will allow us to leverage both the abundant unlabeled data and the limited labeled data more effectively.

6 Acknowledgments

This work was supported by the National Natural Science Foundation of China (No.62302333, 62261045).

References

- [1] Deyu Bo, Xiao Wang, Chuan Shi, Meiqi Zhu, Emiao Lu, and Peng Cui. 2020. Structural Deep Clustering Network. In *Proceedings of the Web Conference*. 1400–1410.
- [2] Karsten M Borgwardt and Hans-Peter Kriegel. 2005. Shortest-Path Kernels on Graphs. In *Proceedings of the Fifth IEEE International Conference on Data Mining* (2005). 74–81.
- [3] Karsten M Borgwardt, Cheng Soon Ong, Stefan Schöner, SVN Vishwanathan, Alex J Smola, and Hans-Peter Kriegel. 2005. Protein Function Prediction via Graph Kernels. *Bioinformatics* 21, suppl_1 (2005), i47–i56.
- [4] Jinyu Cai, Yi Han, Wenzhong Guo, and Jicong Fan. 2024. Deep Graph-Level Clustering Using Pseudo-Label-Guided Mutual Information Maximization Network. *Neural Computing and Applications* 36, 16 (2024), 9551–9566.
- [5] Jinyu Cai, Yunhe Zhang, and Jicong Fan. 2025. Self-Discriminative Modeling for Anomalous Graph Detection. In *Proceedings of the International Conference on Machine Learning* (2025).
- [6] Jinyu Cai, Yunhe Zhang, Jicong Fan, Yali Du, and Wenzhong Guo. 2024. Dual Contrastive Graph-Level Clustering with Multiple Cluster Perspectives Alignment. In *International Joint Conference on Artificial Intelligence* (2024).
- [7] Man-Sheng Chen, Pei-Yuan Lai, De-Zhang Liao, Chang-Dong Wang, and Jian-Huang Lai. 2025. Graph Prompt Clustering. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47, 7 (2025), 5794–5805.
- [8] P D Dobson and A J Doig. 2003. Distinguishing Enzyme Structures from Non-Enzymes Without Alignments. *Journal of Molecular Biology* 330, 4 (2003), 771–783.
- [9] Renxiang Guan, Wenxuan Tu, Zihao Li, Hao Yu, Dayu Hu, Yuzeng Chen, Chang Tang, Qiangqiang Yuan, and Xinwang Liu. 2024. Spatial-Spectral Graph Contrastive Clustering With Hard Sample Mining for Hyperspectral Images. *IEEE Transactions on Geoscience and Remote Sensing* 62 (2024), 1–16.
- [10] T. Gärtner, P. A. Flach, and S. Wrobel. 2003. On Graph Kernels: Hardness Results and Efficient Alternatives. In *Proceedings of the 16th Annual Conference on Computational Learning Theory* (2003). 129–143.
- [11] David K Hammond, Pierre Vandergheynst, and Rémi Gribonval. 2011. Wavelets on Graphs via Spectral Graph Theory. *Applied and Computational Harmonic Analysis* 30, 2 (2011), 129–150.
- [12] Dongxiao He, Yiqi Dong, Xiaobao Wang, Meng Ge, Carl Yang, Di Jin, and Witold Pedrycz. 2025. Unveiling Fine-grained Deceptive Patterns in Multi-modal Fake News: An Explainable Neuro-Symbolic Framework with LVLs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).
- [13] Mengling Hu, Chaochao Chen, Weiming Liu, Xinyi Zhang, Xinting Liao, and Xiaolin Zheng. 2023. Learning Uniform Clusters on Hypersphere for Deep Graph-Level Clustering. *ArXiv E-Prints* (2023), arXiv–2311.
- [14] Qiang Ji, Yanfeng Sun, Junbin Gao, Yongli Hu, and Baocai Yin. 2021. A Decoder-Free Variational Deep Embedding for Unsupervised Clustering. *IEEE Transactions on Neural Networks and Learning Systems* 33, 10 (2021), 5681–5693.
- [15] Di Jin, Jun Yang, Xiaobao Wang, Junwei Zhang, Shuqi Li, and Dongxiao He. 2025. A dynamic knowledge update-driven model with large language models for fake news detection. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*. 3000–3008.
- [16] Fredrik Johansson, Vinay Jethava, Devdatt Dubhashi, and Chiranjib Bhat-tacharyya. 2014. Global Graph Kernels Using Geometric Embeddings. In *International Conference on Machine Learning* (2014), Vol. 32. 694–702.
- [17] Wei Ju, Yiyang Gu, Binqi Chen, Gongbo Sun, Yifang Qin, Xingyuming Liu, Xiao Luo, and Ming Zhang. 2023. Glcc: A General Framework for Graph-Level Clustering. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2023), Vol. 37. 4391–4399.
- [18] Thomas N. Kipf and Max Welling. 2020. Graph Auto-Encoders. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)* (2020).
- [19] Thomas N Kipf and Max Welling. 2016. Variational Graph Auto-Encoders. *ArXiv Preprint ArXiv:1611.07308* (2016).
- [20] Gayan K Kulatilake, Marius Portmann, and Shekhar S Chandra. 2025. SCGC: Self-Supervised Contrastive Graph Clustering. *Neurocomputing* 611 (2025), 128629.
- [21] Shuai Li, Liang Hu, Chengyu Sun, Hongtu Li, and Juncheng Hu. 2026. Enhancing Medical Assistance Through Secure Federated Edge Data Augmentation with Local Differential Privacy in Non-IID Scenarios. *Tsinghua Science and Technology* 31, 2 (2026), 726–744.
- [22] Ke Liang, Yue Liu, Sihang Zhou, Wenxuan Tu, Yi Wen, Xihong Yang, Xiangjun Dong, and Xinwang Liu. 2023. Knowledge Graph Contrastive Learning Based on Relation-Symmetrical Structure. *IEEE Transactions on Knowledge and Data Engineering* 36, 1 (2023), 226–238.
- [23] Ke Liang, Lingyuan Meng, Hao Li, Jun Wang, Long Lan, Miaomiao Li, Xinwang Liu, and Huaimin Wang. 2025. From Concrete to Abstract: Multi-View Clustering on Relational Knowledge. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025), 1–18.
- [24] Ke Liang, Sihang Zhou, Meng Liu, Yue Liu, Wenxuan Tu, Yi Zhang, Liming Fang, Zhe Liu, and Xinwang Liu. 2024. Hawkes-Enhanced Spatial-Temporal Hypergraph Contrastive Learning Based on Criminal Correlations. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2024), Vol. 38. 8733–8741.
- [25] Tianchi Liao, Zhebin Wu, Chuan Chen, Zibin Zheng, and Xiongjun Zhang. 2023. Tensor Completion via Convolutional Sparse Coding with Small Samples-Based Training. *Pattern Recognition* 141 (2023), 109624.
- [26] Tianchi Liao, Ziyue Xu, Qing Hu, Hong-Ning Dai, Huaiwei Huang, Zibin Zheng, and Chuan Chen. 2025. Fedbrb: A Solution to the Small-to-Large Scenario in Device-Heterogeneity Federated Learning. *IEEE Transactions on Mobile Computing* (2025), 1–14.
- [27] Xuan Lin, Lichang Dai, Yafang Zhou, Zu-Guo Yu, Wen Zhang, Jian-Yu Shi, Dong-Sheng Cao, Li Zeng, Haowen Chen, Bosheng Song, et al. 2023. Comprehensive Evaluation of Deep and Graph Learning on Drug-Drug Interactions Prediction. *Briefings in Bioinformatics* 24, 4 (2023), bbad235.
- [28] Jingxin Liu, Renda Han, Wenxuan Tu, Haotian Wang, Junlong Wu, and Jieren Cheng. 2025. Federated Node-Level Clustering Network with Cross-Subgraph Link Mending. In *International Conference on Machine Learning* (2025), Vol. -. -. .
- [29] Jingxin Liu, Xiangyan Tang, Renda Han, Wenxuan Tu, and Ruili Wang. 2025. Adaptive Feature Boosting and Distribution Refinement for Graph Clustering. *Pattern Recognition* (2025), 112309.
- [30] Meng Liu, Ke Liang, Hao Yu, Lingyuan Meng, Siwei Wang, Sihang Zhou, and Xinwang Liu. 2025. Multiview Temporal Graph Clustering. *IEEE Transactions on Neural Networks and Learning Systems* (2025), 1–14.
- [31] Meng Liu, Ke Liang, Yawei Zhao, Wenxuan Tu, Sihang Zhou, Xinbiao Gan, Xinwang Liu, and He Kunlun. 2024. Self-Supervised Temporal Graph Learning with Temporal and Structural Intensity Alignment. *IEEE Transactions on Neural Networks and Learning Systems* (2024).
- [32] Meng Liu, Yue Liu, Ke Liang, Wenxuan Tu, Siwei Wang, Sihang Zhou, and Xinwang Liu. 2024. Deep Temporal Graph Clustering. In *The 12th International Conference on Learning Representations* (2024).
- [33] Suyuan Liu, Ke Liang, Zhibin Dong, Siwei Wang, Xihong Yang, Sihang Zhou, En Zhu, and Xinwang Liu. 2024. Learn from View Correlation: An Anchor Enhancement Strategy for Multi-View Clustering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024). 26151–26161.
- [34] Zhaowei Liu, Sheng Liu, Weiqing Yan, Peng Song, Yongchao Song, and Rufe Gao. 2026. ATGFB-MFF: Adaptive Text-Guided Fiber Bundle Feature Fusion with LLMs for Multimodal Sentiment Analysis and Emotion Recognition in Conversations. In *Proceedings of the ACM on Web Conference 2026*.
- [35] Christopher Morris, Nils M Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. TUDataset: A Collection of Benchmark Datasets for Learning with Graphs. (2020).
- [36] Marion Neumann, Plinio Moreno, Laura Antanas, Roman Garnett, and Kristian Kersting. 2013. Graph Kernels for Object Category Prediction in Task-Dependent Robot Grasping. *Online Proceedings of the Eleventh Workshop on Mining and Learning with Graphs* (2013), 0–6.
- [37] Xin Peng, Jieren Cheng, Xiangyan Tang, Jingxin Liu, and Jiahua Wu. 2023. Dual Contrastive Learning Network for Graph Clustering. *IEEE Transactions on Neural Networks and Learning Systems* (2023).
- [38] Fan-Yun Sun, Jordan Hoffmann, Vikas Verma, and Jian Tang. 2020. InfoGraph: Unsupervised and Semi-Supervised Graph-Level Representation Learning via Mutual Information Maximization. In *International Conference on Learning Representations (ICLR)* (2020).
- [39] Jeffrey J Sutherland, Lee A O'brien, and Donald F Weaver. 2003. Spline-Fitting with a Genetic Algorithm: A Method for Developing Classification Structure-Activity Relationships. *Journal of Chemical Information and Computer Sciences* 43, 6 (2003), 1906–1915.
- [40] Yue Tan, Yixin Liu, Guodong Long, Jing Jiang, Qinghua Lu, and Chengqi Zhang. 2023. Federated Learning on Non-lid Graphs via Structural Knowledge Sharing. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2023). 9953–9961.
- [41] Chang Tang, Zhenglai Li, Jun Wang, Xinwang Liu, Wei Zhang, and En Zhu. 2022. Unified One-Step Multi-View Spectral Clustering. *IEEE Transactions on Knowledge and Data Engineering* 35, 6 (2022), 6449–6460.
- [42] Wenxuan Tu, Sihang Zhou, Xinwang Liu, Xifeng Guo, Zhiping Cai, En Zhu, and Jieren Cheng. 2021. Deep Fusion Clustering Network. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2021), Vol. 35. 9978–9987.
- [43] Wenxuan Tu, Sihang Zhou, Xinwang Liu, Yue Liu, Zhiping Cai, En Zhu, Chang-wang Zhang, and Jieren Cheng. 2022. Initializing Then Refining: A Simple Graph Attribute Imputation Network. In *Proceedings of the International Joint Conference on Artificial Intelligence* (2022). 3494–3500.
- [44] Rianne Van Den Berg, N Kipf Thomas, and Max Welling. 2017. Graph Convolutional Matrix Completion. (2017), 9 pages.
- [45] P. Velickovic and et al. 2018. Deep Graph Infomax. In *Proceedings of the 35th International Conference on Machine Learning (ICML)* (2018).
- [46] Petar Velickovic, Guillem Cucurull, Alberto Casanova, Javier Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph Attention Networks. In *Proceedings of the 6th International Conference on Learning Representations (ICLR)* (2018).
- [47] Xiaobao Wang, Yiqi Dong, Di Jin, Yawen Li, Longbiao Wang, and Jianwu Dang. 2023. Augmenting affective dependency graph via iterative incongruity graph learning for sarcasm detection. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 4702–4710.

- [48] Xiaobao Wang, Meng Ge, Lingshan Li, Di Jin, Kai He, and Erik Cambria. 2025. Global Entity Relationship Enhancement Network for Multimodal Sarcasm Detection. *IEEE Transactions on Affective Computing* (2025), 1–14.
- [49] Xiaobao Wang, Ruoxiao Sun, Yujun Zhang, Bingdao Feng, Dongxiao He, Luzhi Wang, and Di Jin. [n. d.]. Stealthy Yet Effective: Distribution-Preserving Backdoor Attacks on Graph Classification. In *Proceedings of the Annual Conference on Neural Information Processing Systems*.
- [50] Scott White and Padhraic Smyth. 2005. A Spectral Clustering Approach to Finding Communities in Graphs. In *Proceedings of the 2005 SIAM International Conference on Data Mining*. 274–285.
- [51] Yongheng Xing, Liang Hu, Xinqi Du, Zhiqi Shen, Juncheng Hu, and Feng Wang. 2024. CCDF-TAP: a Context-Aware Conflict Detection Framework for IoT Trigger-Action Programming with Graph Neural Network. *IEEE Internet of Things Journal* 11, 19 (2024), 31534–31544.
- [52] Yongheng Xing, Liang Hu, Xinqi Du, Zhiqi Shen, Juncheng Hu, and Feng Wang. 2024. A Privacy-Preserving Federated Graph Learning Framework for Threat Detection in IoT Trigger-Action Programming. *Expert Systems with Applications* 255 (2024), 124724.
- [53] Hongteng Xu, Jiachang Liu, Dixin Luo, and Lawrence Carin. 2022. Representing Graphs via Gromov-Wasserstein Factorization. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 1 (2022), 999–1016.
- [54] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How Powerful Are Graph Neural Networks?. In *Proceedings of the 7th International Conference on Learning Representations* (2018).
- [55] Zhiming Xu, Xiao Huang, Yue Zhao, Yushun Dong, and Jundong Li. 2022. Contrastive Attributed Network Anomaly Detection with Data Augmentation. In *PAKDD* (2022). 444–457.
- [56] Xifeng Yan, Hong Cheng, Jiawei Han, and Philip S Yu. 2008. Mining Significant Graph Patterns by Leap Search. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 433–444.
- [57] Yuning You, Tianlong Chen, Yang Shen, and Zhangyang Wang. 2021. Graph Contrastive Learning Automated. In *International Conference on Machine Learning* (2021). 12121–12132.
- [58] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph Contrastive Learning with Augmentations. *Proceedings of the Conference on Neural Information Processing Systems*, 5812–5823.
- [59] Si Zhang, Hanghang Tong, Jiejun Xu, and Ross Maciejewski. 2019. Graph convolutional networks: a comprehensive review. *Computational Social Networks* 6, 1 (2019), 1–23.

Appendix Abstract

This appendix includes the following contents:

- (1) Related Work.
- (2) Details of evaluation metric computation.
- (3) Details of UGCN implementation.
- (4) Details of benchmark datasets.
- (5) Details of baseline methods.
- (6) More detailed experimental results.

A Related Work

A.1 Node-level Graph Clustering

Node-level graph clustering aims to group similar nodes in a graph based on their attributes or structural relationships. Early node-level clustering approaches are primarily based on spectral clustering [50]. These methods rely heavily on hand-crafted features and are computationally expensive for large-scale graphs. With the rise of Graph Neural Networks (GNNs), node-level clustering has seen a paradigm shift, such as GCN [19], GraphSAGE [11], and GAT [45], which learn node representations through message-passing mechanisms, which capture both node features and local graph structure. For example, SDCN [1] integrates structural information from graphs to improve clustering accuracy. DFCN [42], employs a three self-supervision mechanism to optimize node-level representations with the added benefit of improving clustering performance. Recent advancements have introduced contrastive learning [46] techniques, which learn to optimize node embeddings with respect

to positive and negative node pairs. For example, DCLN [37] mitigates the issues of over-smoothing and allows the model to learn more discriminative node representations. More recently, SCGC [20] dynamically fuses discriminative node representations and refines soft cluster assignments, addressing the over-smoothing issues. Its enhanced version, SCGC*, incorporates an expressive Influence metric and IAC loss, achieving superior performance and efficiency with reduced model parameters. Similarly, ABR [29] captures key graph structures and node attributes through a local-global feature boosting approach, while also refining the target distribution via a dynamic strategy that emphasizes the importance of soft assignments. Furthermore, node-level clustering in federated learning settings has also emerged as a promising research direction. FedNCN [28] explores decentralized clustering paradigms in which graph data are distributed across multiple clients, ensuring privacy preservation.

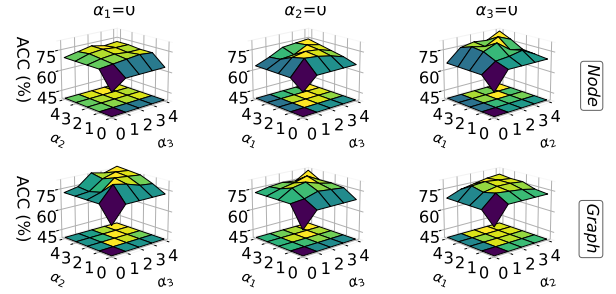


Figure 8: ACC performance for node- and graph-level tasks on BZR dataset under various ratios of hyper-parameters α_1 , α_2 , and α_3 from [0:0,4:4].

A.2 Graph-level Clustering

Graph-level clustering, as a key challenge in graph-structured data analysis, aims to group entire graphs into meaningful clusters based on their latent patterns. This task has gained significant attention due to its applications in molecular biology, social networks, and chemistry. One of the earlier approaches in graph-level clustering relied on traditional methods like spectral clustering [50] and graph kernel [10, 36]. The former uses eigenvalue decomposition to embed graphs into a lower-dimensional space, and the latter calculates the similarity between graphs in Hilbert space. While these methods are effective, they often struggle to handle large-scale graphs and graph heterogeneity, limiting their scalability and generalizability. With the rise of deep contrastive learning, GNNs have revolutionized graph-level clustering tasks. Models such as InfoGraph [38], GraphCL [58], JOAO [57], GFW [53], and GIN [54] have enabled the learning of powerful graph representations. These models extract and pool node attributes to obtain global graph-level representations, which can then be used for clustering tasks. The first significant approach is GLCC [17], which improves graph-level clustering by maximizing the similarity between graphs that belong to the same cluster while minimizing the similarity between graphs from different clusters. Moreover, DGLC [4] learn discriminative graph-level representations by exploring the graph’s inherent structure and content. Another key approach, UDGC [13], maps the graph manifold to a hypersphere for clustering. Moreover, DCGLC

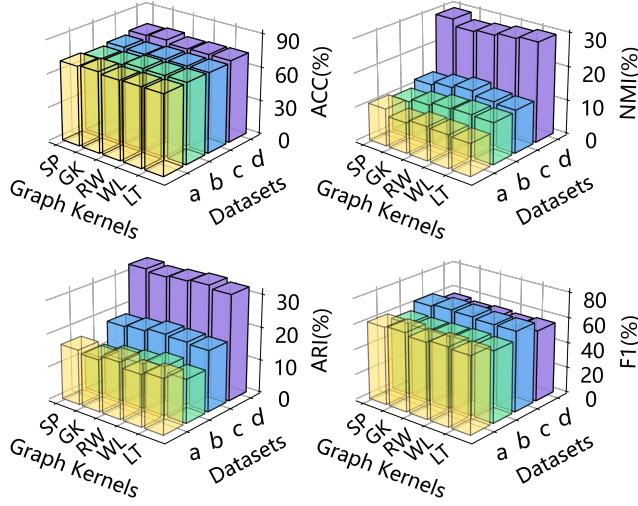


Figure 9: Clustering performance under different graph kernels. “a”, “b”, “c”, and “d” denote node-level clustering on DHFR, graph-level clustering on DHFR, graph-level clustering on PROTEINS, node-level clustering on PROTEINS, respectively, reporting ACC, NMI, ARI, and F1 scores.

[6] extends to DGLC, which introduces the Euclidean-based and subspace-based cluster heads to capture the cluster information from different cluster perspectives. Very recently, GPC [7] utilizes mutual information maximization to pretrain, followed by fine-tuning, where learnable prompt vectors adapt the pretrained model to target graph datasets.

B Computation of Evaluation Metrics

We employ four standard metrics for graph clustering performance evaluation: **Accuracy (ACC)**, **Normalized Mutual Information (NMI)**, **Adjusted Rand Index (ARI)**, and **F1 Score**. These metrics provide a comprehensive assessment of clustering quality. Their computation details are as follows: **ACC**: Accuracy measures the proportion of correctly clustered data points relative to the total number of data points. Given the true cluster labels $\{y_i\}$ and predicted cluster labels $\{\hat{y}_i\}$, the accuracy is defined as: $ACC = \frac{\sum_{i=1}^n \mathbb{I}(y_i = \hat{y}_i)}{n}$ where $\mathbb{I}(y_i = \hat{y}_i)$ is the indicator function that equals 1 if $y_i = \hat{y}_i$ (i.e., the true and predicted labels match), and 0 otherwise. **NMI**: NMI measures the amount of information shared between the true and predicted cluster assignments. It is defined as: $NMI = \frac{I(y, \hat{y})}{\max(H(y), H(\hat{y}))}$ where $I(y, \hat{y})$ is the mutual information between the true labels y and the predicted labels $\hat{y}$, and $H(y)$ and $H(\hat{y})$ are the entropies of the true and predicted labels, respectively. NMI is bounded between 0 and 1, where 1 indicates perfect alignment. **ARI**: ARI measures the similarity between two data clusterings, adjusted for chance. It is defined as: $ARI = \frac{RI - \mathbb{E}[RI]}{\max(RI) - \mathbb{E}[RI]}$ where RI is the Rand Index, and $\mathbb{E}[RI]$ is the expected value of the Rand Index under random label assignments. ARI ranges from -1 (indicating completely dissimilar clusters) to 1 (indicating perfect similarity), with 0 corresponding to random clustering. **F1**: The F1 score is the harmonic mean of precision and

recall, providing a balanced measure of clustering performance. Given the true positive (TP), false positive (FP), and false negative (FN) counts, the F1 score is defined as: $F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ where: $\text{Precision} = \frac{TP}{TP + FP}$, $\text{Recall} = \frac{TP}{TP + FN}$. The F1 score provides a good balance between precision and recall, making it particularly useful in clustering tasks where both false positives and false negatives are of concern.

C Implementation Details

C.1 System Resources

We train the models on a high-performance computing setup, utilizing a single NVIDIA GeForce RTX 4090 GPU for accelerated processing. The system is powered by an Intel Core i9-12900K 12th-generation CPU, offering robust multi-core performance, and is equipped with 64GB RAM, ensuring efficient handling of large-scale computations and smooth parallel processing during training. All experiments are implemented in Python using PyTorch (version 2.4.0, CUDA 11.7) and PyTorch Geometric (version 2.1.0), running on the Windows 10 Operating System. The Python environment is managed via Anaconda (version 3.9).

C.2 Parameter Settings

For optimization, we employ the Adam optimizer with a learning rate of 0.001, decaying at a rate of $1e - 4$, and utilize a batch size of 256. We leverage LT as the graph kernel function. σ is the ReLU activation function. T is set to 0.35. α_1 , α_2 , and α_3 are fixed to 1, 1, and 1, respectively. μ is fixed to 0.5. τ is fixed to 1. Following common practice, we set the number of layers of the encoder to 4, with the hidden feature dimension fixed at 10 for the single graph dataset and 4 for multi-graph datasets. The model is trained for 100 and 30 epochs for the node-level and graph tasks, respectively, ensuring sufficient convergence for each task. To ensure the robustness and reliability of the results, each experiment is independently repeated five times, with results averaged to mitigate potential variability.

D Details of Benchmark Datasets

In this paper, six datasets from different domains are employed as benchmark datasets. All graph benchmarks are obtained from the publicly available TUDataset collection [35]. Detailed statistics of these datasets are summarized in Table 1, including the total number of graphs, the average numbers of nodes and edges, the number of node and graph classes, node classes, and the data types. The detailed information for these datasets is as follows:

- **COX2** [39]: This dataset contains molecular graphs of inhibitors. Nodes and edges denote atoms and chemical bonds.
- **BZR** [39]: This dataset contains molecular graphs of receptor ligands. Nodes and edges denote atoms and chemical bonds.
- **DHFR** [39]: This dataset consists of molecular graphs of dihydrofolate reductase inhibitors. Each graph encodes a molecule, with nodes representing atoms and edges representing chemical bonds. Node features include atom types.
- **PROTEINS** [3]: This dataset contains protein graphs where nodes represent secondary structure elements and edges indicate neighborhood relationships.

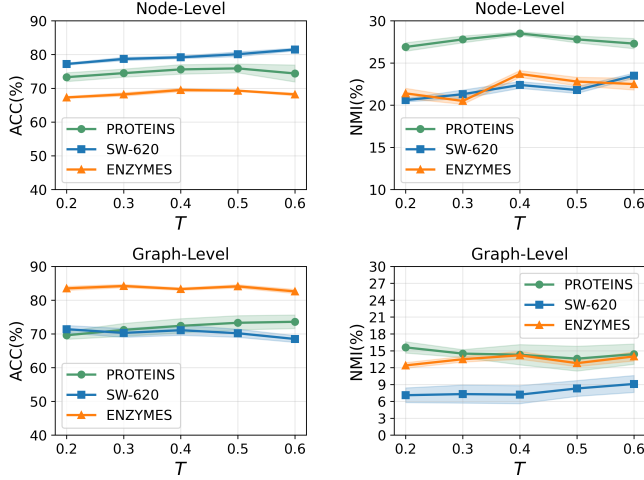


Figure 10: Sensitivity analysis of hyper-parameters T for node-level and graph-level tasks on three datasets, reporting ACC and NMI metrics.

Table 3: Performance comparison results (expressed as mean $\pm$ standard deviation) of the graph-level clustering task on ENZYMES and PC-3 datasets. Bold and underlined represent the highest and second-highest performances, respectively.

Node-level Clustering								
Methods	ENZYMES				PC-3			
	ACC	NMI	ARI	F1	ACC	NMI	ARI	F1
AE	50.0 $\pm$ 0.1	15.0 $\pm$ 0.2	11.0 $\pm$ 0.2	40.1 $\pm$ 0.3	73.0 $\pm$ 0.3	0.2 $\pm$ 0.0	0.0 $\pm$ 0.0	18.8 $\pm$ 0.0
GAE	47.1 $\pm$ 0.3	9.8 $\pm$ 0.4	10.5 $\pm$ 0.6	41.2 $\pm$ 0.4	68.4 $\pm$ 0.5	1.6 $\pm$ 0.0	2.2 $\pm$ 0.3	17.5 $\pm$ 0.3
DEC	51.6 $\pm$ 0.5	14.6 $\pm$ 0.2	13.4 $\pm$ 0.4	41.5 $\pm$ 0.6	55.6 $\pm$ 0.3	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	16.4 $\pm$ 0.5
SDCN	64.0 $\pm$ 0.2	20.0 $\pm$ 0.3	20.0 $\pm$ 0.3	44.0 $\pm$ 0.4	77.7 $\pm$ 0.8	8.7 $\pm$ 0.4	15.2 $\pm$ 0.8	30.9 $\pm$ 1.0
DFCN	58.7 $\pm$ 0.8	17.6 $\pm$ 0.5	11.4 $\pm$ 1.4	45.9 $\pm$ 1.0	79.2 $\pm$ 0.3	10.3 $\pm$ 1.1	8.7 $\pm$ 0.5	37.6 $\pm$ 2.3
DCLN	56.9 $\pm$ 2.6	23.4 $\pm$ 1.0	24.5 $\pm$ 1.3	42.6 $\pm$ 1.3	57.8 $\pm$ 0.8	12.4 $\pm$ 1.3	14.2 $\pm$ 1.2	27.5 $\pm$ 1.0
SCGC	58.2 $\pm$ 0.6	22.5 $\pm$ 0.5	21.7 $\pm$ 0.6	43.6 $\pm$ 1.4	62.4 $\pm$ 1.1	10.5 $\pm$ 0.7	13.1 $\pm$ 0.9	27.5 $\pm$ 1.5
ABR	64.3 $\pm$ 0.6	17.8 $\pm$ 0.6	16.6 $\pm$ 0.4	46.7 $\pm$ 0.3	79.8 $\pm$ 0.4	15.2 $\pm$ 0.7	18.9 $\pm$ 0.6	34.1 $\pm$ 0.3
OURS	68.6$\pm$0.3	21.4$\pm$0.5	21.6$\pm$0.3	51.5$\pm$0.4	82.5$\pm$0.6	19.4$\pm$0.6	22.6$\pm$0.7	64.6$\pm$0.5
Graph-level Clustering								
RW	22.0 $\pm$ 0.0	2.5 $\pm$ 0.0	1.6 $\pm$ 0.0	5.3 $\pm$ 0.0	50.0 $\pm$ 0.0	1.7 $\pm$ 0.0	2.1 $\pm$ 0.0	8.6 $\pm$ 0.0
LT	17.0 $\pm$ 0.0	0.4 $\pm$ 0.0	0.0 $\pm$ 0.0	5.7 $\pm$ 0.0	51.6 $\pm$ 0.2	1.5 $\pm$ 0.4	1.6 $\pm$ 1.0	11.9 $\pm$ 0.0
InfoGraph	22.4 $\pm$ 0.3	2.4 $\pm$ 0.6	1.2 $\pm$ 0.2	6.5 $\pm$ 0.4	54.5 $\pm$ 0.1	3.8 $\pm$ 0.2	0.2 $\pm$ 0.1	11.8 $\pm$ 0.3
GraphCL	25.8 $\pm$ 0.1	0.5 $\pm$ 0.2	1.9 $\pm$ 0.5	7.3 $\pm$ 0.2	55.6 $\pm$ 0.3	3.7 $\pm$ 0.8	2.9 $\pm$ 0.6	13.4 $\pm$ 1.3
JOAO	21.4 $\pm$ 0.4	1.6 $\pm$ 0.3	0.9 $\pm$ 0.2	8.5 $\pm$ 0.9	51.9 $\pm$ 0.2	2.7 $\pm$ 0.5	5.6 $\pm$ 0.7	15.21 $\pm$ 0.8
GFW	29.6 $\pm$ 0.2	6.2 $\pm$ 0.6	3.1 $\pm$ 1.8	12.7 $\pm$ 0.9	52.7 $\pm$ 0.8	3.3 $\pm$ 1.1	0.0 $\pm$ 0.0	26.4 $\pm$ 0.9
GLCC	19.9 $\pm$ 1.1	2.4 $\pm$ 0.2	0.1 $\pm$ 0.1	8.7 $\pm$ 1.2	54.63 $\pm$ 0.3	6.1 $\pm$ 0.3	5.9 $\pm$ 1.4	15.5 $\pm$ 0.8
UDGC	18.7 $\pm$ 0.6	0.5 $\pm$ 1.1	1.4 $\pm$ 2.2	42.2 $\pm$ 1.5	53.6 $\pm$ 1.8	2.4 $\pm$ 1.4	3.2 $\pm$ 0.8	13.2 $\pm$ 0.3
DGLC	23.2 $\pm$ 0.4	4.9 $\pm$ 0.1	2.5 $\pm$ 3.4	14.1 $\pm$ 0.6	52.4 $\pm$ 0.6	3.6 $\pm$ 0.2	4.2 $\pm$ 0.3	19.4 $\pm$ 0.3
DCGLC	<u>28.4$\pm$1.1</u>	<u>6.2$\pm$0.2</u>	<u>3.8$\pm$0.4</u>	<u>15.2$\pm$0.4</u>	<u>59.8$\pm$1.3</u>	<u>9.7$\pm$1.2</u>	<u>8.6$\pm$1.0</u>	<u>24.5$\pm$1.0</u>
OURS	83.5$\pm$0.4	12.4$\pm$0.6	8.4$\pm$0.0	76.2$\pm$1.3	63.4$\pm$0.5	16.5$\pm$0.2	10.8$\pm$0.4	26.5$\pm$0.7

- **DD** [8]: This dataset contains protein structure graphs. Nodes are amino acids, and edges denote spatial proximity. The classification task is to categorize proteins according to their structural or functional class.
- **SW-620** [56]: This dataset contains cell line graphs. The graphs represent cell signaling pathways, with nodes corresponding to molecules (e.g., proteins, small molecules), and edges indicating interactions between molecules.

- **PC-3** [56]: This dataset contains molecular graphs, which related to a prostate cancer cell line. Each graph corresponds to a chemical compound with nodes representing atoms.

E Details of Baseline Methods

To comprehensively evaluate our proposed approach, we compare it with a range of baseline methods across both node-level and graph-level clustering tasks. These tasks are categorized into classic methods, early deep methods, and advanced deep methods, each representing different stages of development in graph clustering techniques. For node-level tasks, **classic methods** include:

- **AE (Autoencoders)** [18]: A method that uses an encoder-to-decoder architecture to learn low-dimensional node representations by reconstructing node features.
- **GCN (Graph Convolutional Network)** [59]: A pioneering deep learning model that uses convolutional operations over a graph to aggregate information from neighboring nodes.
- **DEC** [14]: A method that simultaneously learns feature representations and cluster assignments using a variational auto-encoder framework.

For graph-level tasks, **classic methods** include:

- **RW Graph Kernel** [2]: The RW Graph Kernel captures similarities between graphs using the shortest path information based on random walks.
- **LT Graph Kernel** [16]: The LT Graph Kernel captures global graph properties by Lovász number.

Early deep methods include:

- **InfoGraph** [38]: InfoGraph learns graph-level representations by maximizing mutual information between graph-level and substructure-level representations.
- **GraphCL** [58]: GraphCL employs four types of augmentations to incorporate various priors, enabling unsupervised graph representation learning.
- **JOAO** [57]: JOAO presents a unified bi-level optimization framework to automate, adapt, and dynamically select data augmentations for graph contrastive learning, addressing the limitations imposed by manual augmentation tuning.
- **GFW** [53]: GFW learns graph embeddings in a flexible and interpretable manner by reconstructing each graph as a weighted combination of shared graph factors using the Gromov-Wasserstein discrepancy.

F More Detailed Experimental Results

Here, we present more detailed experimental results, including:

- (1) Performance comparison results of node- and graph-level tasks on the ENZYMES and PC-3 datasets, reporting four metrics, as demonstrated in Table 3.
- (2) Performance comparison of different loss ratios for node- and graph-level tasks on BZR dataset, as shown in Fig. 8.
- (3) Performance comparison of various graph kernels for node-level and graph-level tasks on DHFR and PROTEINS datasets, evaluated using four metrics, as presented in Fig. 9.
- (4) Hyperparameter sensitivity analysis of hyperparameter T for node-level and graph-level tasks on the PROTEINS, ENZYMES, and SW-620 datasets, reporting ACC and NMI metrics, as illustrated in Fig. 10.